

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ № 1

з дисципліни “Професійний Python”

на тему “Організація професійного Python-проєкту та Git”

Виконав

студент 3 курсу

групи ФЕП-33

Рак Богдан Ігорович

Перевірив

Професор кафедри ОІТ

Свелеба Сергій Андрійович

Львів – 2026

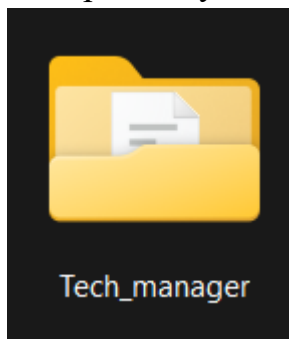
Мета: Ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки, формувати структуру проєкту за принципом src-layout, працювати з модулями та пакетами Python, створювати файл pyproject.toml, використовувати Git для контролю версій та оформлювати базову документацію проєкту.

1. Встановити Python.

```
C:\Users\w1nzeen>py
Python 3.13.15 (tags/v3.13.15:4061bc4, Aug  5 2026, 13:05:39) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> |
```

2. Створити каталог проєкту.

Створив теку для проєкту



3. Створити .venv, 4. Активувати virtual environment.

```
PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> python -m venv .venv
PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> .\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> |
```

Створив віртуальне середовище та активував його.

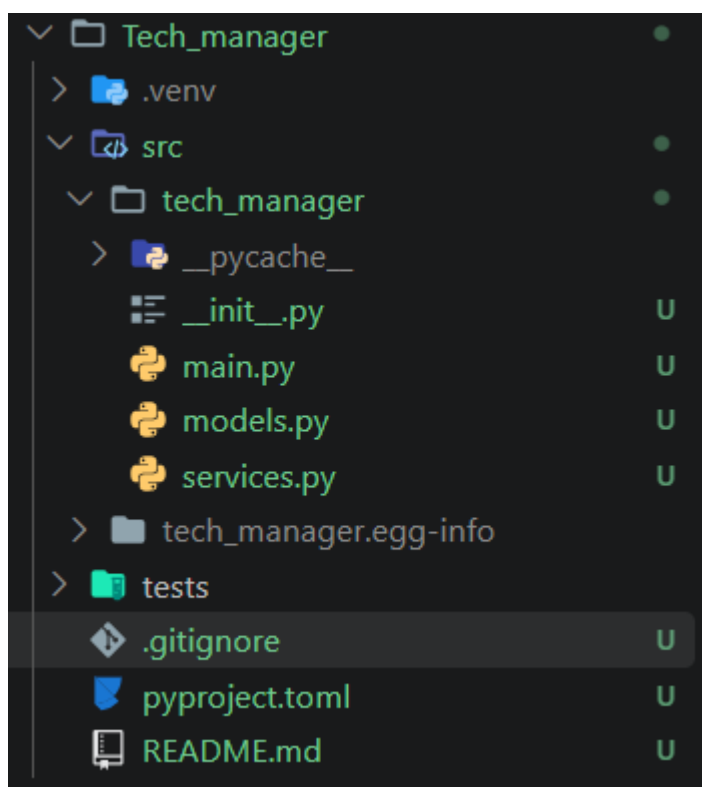
5. Ініціалізувати Git repository.

```
(.venv) PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> git init
Initialized empty Git repository in C:/Users/w1nzeen/OneDrive/Робочий стіл/SI 3 term/prof python/Tech_manager/.git/
(.venv) PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> |
```

6. Створити .gitignore.

```
.gitignore U X
prof python > Tech_manager > .gitignore
1  .venv/
2  __pycache__/
3  *.pyc
4  .pytest_cache/
5  .idea/
6  .vscode/
7  dist/
8  build/
9  *.egg-info/
10 .env
11 |
```

7. Створити src-layout.



8. Створити package.

```
(.venv) PS C:\Users\winzeen\OneDrive\Робочий стіл\SI 3 term\prof python\tech_manager> python -m pip install -e .
Obtaining file:///C:/Users/winzeen/OneDrive/%D0%A0%D0%BE%D0%B1%D0%BE%D1%87%D0%B8%D0%B9%20%D1%81%D1%82%D1%96%D0%BB/SI%203%20term/prof%20python/Tec
h_manager
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Building wheels for collected packages: tech-manager
Building editable for tech-manager (pyproject.toml) ... done
Created wheel for tech-manager: filename=tech_manager-0.1.0-0.editable-py3-none-any.whl size=1604 sha256=679853bcf2cbb3d10216176901d0917e8b1ee8
357d23364b133280949691c73e
Stored in directory: C:\Users\winzeen\AppData\Local\Temp\pip-ephem-wheel-cache-ee6aeb3u\wheels\8c\24\d0\d35d8f318125127c82fe56d5fdae0c8d95f1dfa
f8582fed618
Successfully built tech-manager
Installing collected packages: tech-manager
Attempting uninstall: tech-manager
Found existing installation: tech-manager 0.1.0
Uninstalling tech-manager-0.1.0:
Successfully uninstalled tech-manager-0.1.0
Successfully installed tech-manager-0.1.0
(.venv) PS C:\Users\winzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager>
```

9. Створити models.

```
models.py U X
prof python > Tech_manager > src > tech_manager > models.py > Tech > name
1 from dataclasses import dataclass
2
3 @dataclass
4 class Tech:
5     inventory_number: str
6     type: str
7     vendor: str
8     model: str
9     price: float
10
11     @property
12     def name(self) -> str:
13         return f"{self.type}: {self.vendor} {self.model}"
```

10. Створити services.py.

```
services.py U X
prof python > Tech_manager > src > tech_manager > services.py > price_of_all_tech
1 from tech_manager.models import Tech
2
3 def search_by_model(tech_list: list[Tech], model: str) -> list[Tech]:
4     return [tech for tech in tech_list if tech.model.lower() == model.lower()]
5
6
7 def sort_by_vendor(tech_list: list[Tech]) -> list[Tech]:
8     return sorted(tech_list, key=lambda tech: tech.vendor.lower())
9
10
11 def avg_price(tech_list: list[Tech]) -> float:
12     if not tech_list:
13         return 0.0
14     total_price = sum(tech.price for tech in tech_list)
15     return total_price / len(tech_list)
16
17 def find_most_expensive_tech(tech_list: list[Tech]) -> Tech:
18     if not tech_list:
19         return None
20     return max(tech_list, key=lambda tech: tech.price)
21
22 def filter_by_price_range(tech_list: list[Tech], min_price: float, max_price: float) -> list[Tech]:
23     return [tech for tech in tech_list if min_price <= tech.price <= max_price]
24
25 def price_of_all_tech(tech_list: list[Tech]) -> float:
26     return sum(tech.price for tech in tech_list)
```

11. Створити main.py.

```
main.py U x
prof python > Tech_manager > src > tech_manager > main.py > main
1 from tech_manager.models import Tech
2 from tech_manager.services import (search_by_model,
3                                   sort_by_vendor,
4                                   avg_price,
5                                   find_most_expensive_tech,
6                                   filter_by_price_range,
7                                   price_of_all_tech)
8
9 def create_demo_tech_list() -> list[Tech]:
10     return [
11         Tech(inventory_number="001", type="Laptop", vendor="Dell", model="XPS 13", price=999.99),
12         Tech(inventory_number="002", type="Smartphone", vendor="Apple", model="iPhone 13", price=799.99),
13         Tech(inventory_number="003", type="Tablet", vendor="Samsung", model="Galaxy Tab S7", price=649.99),
14         Tech(inventory_number="004", type="Laptop", vendor="HP", model="Spectre x360", price=1199.99),
15         Tech(inventory_number="005", type="Smartphone", vendor="Google", model="Pixel 6", price=599.99),
16     ]
17
18 def print_tech_list(tech_list: list[Tech]) -> None:
19     for tech in tech_list:
20         print(f"Inventory Number: {tech.inventory_number}, Type: {tech.type}, Vendor: {tech.vendor}, Model: {tech.model}, Price: $")
21
22
```

12. Реалізувати свій індивідуальний варіант.

Оскільки я 13 в списку, я реалізував 13 варіант.

Варіант 13. Система обліку комп'ютерної техніки

Для пристрою:

- інвентарний номер;
- тип;
- виробник;
- модель;
- вартість.

Реалізувати:

- пошук;
- фільтрацію за виробником;
- середню вартість;
- найдорожчий пристрій;
- загальну вартість обладнання.

13. Додати type hints.

```
prof python > Tech_manager > src > tech_manager > services.py > price_of_all_tech
1  from tech_manager.models import Tech
2
3  def search_by_model(tech_list: list[Tech], model: str) -> list[Tech]:
4      return [tech for tech in tech_list if tech.model.lower() == model.lower()]
5
6
7  def sort_by_vendor(tech_list: list[Tech]) -> list[Tech]:
8      return sorted(tech_list, key=lambda tech: tech.vendor.lower())
9
10
11 def avg_price(tech_list: list[Tech]) -> float:
12     if not tech_list:
13         return 0.0
14     total_price = sum(tech.price for tech in tech_list)
15     return total_price / len(tech_list)
16
17 def find_most_expensive_tech(tech_list: list[Tech]) -> Tech:
18     if not tech_list:
19         return None
20     return max(tech_list, key=lambda tech: tech.price)
21
22 def filter_by_price_range(tech_list: list[Tech], min_price: float, max_price: float) -> list[Tech]:
23     return [tech for tech in tech_list if min_price <= tech.price <= max_price]
24
25 def price_of_all_tech(tech_list: list[Tech]) -> float:
26     return sum(tech.price for tech in tech_list)
```

Додав type hints для функцій.

14. Створити pyproject.toml.

```
pyproject.toml
prof python > Tech_manager > pyproject.toml
1  [build-system]
2  requires = ["setuptools>=70"]
3  build-backend = "setuptools.build_meta"
4
5  [project]
6  name = "tech-manager"
7  version = "0.1.0"
8  description = "Tech management laboratory project"
9  requires-python = ">=3.11"
10
11 [project.scripts]
12 Tech-manager = "tech_manager.main:main"
13
```

15. Встановити package у editable mode.

```
(.venv) PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> python -m pip install -e .
Obtaining file:///C:/Users/w1nzeen/OneDrive/%D0%A0%D0%BE%D0%B1%D0%BE%D1%87%D0%B8%D0%B9%20%D1%81%D1%82%D1%96%D0%BB/SI%203%20term/prof%20python/Tec
h_manager
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Building wheels for collected packages: tech-manager
Building editable for tech-manager (pyproject.toml) ... done
Created wheel for tech-manager: filename=tech_manager-0.1.0-0.editable-py3-none-any.whl size=1604 sha256=e20fdc7c0a3af7aab68f9d8a3a4f7a61519c57
9bf5d0a5392773d71e41e0fa50
Stored in directory: C:\Users\w1nzeen\AppData\Local\Temp\pip-ephem-wheel-cache-4c9ehi1g\wheels\8c\24\d0\d35d8f318125127c82fe56d5fdae0c8d95f1dfa
f8582fed618
Successfully built tech-manager
Installing collected packages: tech-manager
Attempting uninstall: tech-manager
Found existing installation: tech-manager 0.1.0
Uninstalling tech-manager-0.1.0:
Successfully uninstalled tech-manager-0.1.0
Successfully installed tech-manager-0.1.0
(.venv) PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager>
```

16. Перевірити запуск.

```
(.venv) PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> tech-manager

Tech Manager Menu:
1. View all tech items
2. Search by model
3. Sort by vendor
4. Calculate average price
5. Find the most expensive tech item
6. Filter by price range
7. Calculate total price of all tech items
8. Exit
Enter your choice: 1

All Tech Items:
Inventory Number: 001, Type: Laptop, Vendor: Dell, Model: XPS 13, Price: $999.99
Inventory Number: 002, Type: Smartphone, Vendor: Apple, Model: iPhone 13, Price: $799.99
Inventory Number: 003, Type: Tablet, Vendor: Samsung, Model: Galaxy Tab S7, Price: $649.99
Inventory Number: 004, Type: Laptop, Vendor: HP, Model: Spectre x360, Price: $1199.99
Inventory Number: 005, Type: Smartphone, Vendor: Google, Model: Pixel 6, Price: $599.99

Tech Manager Menu:
1. View all tech items
2. Search by model
3. Sort by vendor
4. Calculate average price
5. Find the most expensive tech item
6. Filter by price range
7. Calculate total price of all tech items
8. Exit
Enter your choice: 2
Enter the model to search for: Galaxy Tab S7

Search Results:
Inventory Number: 003, Type: Tablet, Vendor: Samsung, Model: Galaxy Tab S7, Price: $649.99
```

17. Створити README.md.

```
prof python > Tech_manager > README.md > abc # Tech Manager > abc ## Description
1  # Tech Manager
2
3  Laboratory project for the course
4  "Professional Python".
5
6  ## Description
7
8  Console application for managing technic.
9
10 ## Requirements
11
12 Python 3.11+
13
14 ## Installation
15
16 python -m venv .venv
17
18 Activate the virtual environment.
19
20 python -m pip install -e .
21
22 ## Run
23
24 python -m tech_manager.main
25
26 or
27
28 tech-manager
29
30 ## Project structure
31
32 src/tech_manager/models.py
33 Data models.
34
35 src/tech_manager/services.py
36 Business logic.
```

18. Зробити не менше трьох Git commits.

```
• (.venv) PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> git log --oneline
5ca2695 (HEAD -> main, origin/main) Created .toml file
513968d Created .gitignore file
59d0217 Created README file
f6bfc80 Created init file
3257f66 Created services file
d12f10f Created models file
f432e2e Created main file
○ (.venv) PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager>
```


19.Перевірити git status.

```
• (.venv) PS C:\Users\winzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
○ (.venv) PS C:\Users\winzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> █
```

Контрольні запитання

1. Що таке Python-проект?

Це організований набір файлів і каталогів (вихідний код, конфігурація, тести, документація), об'єднаних спільною метою та структурою, який можна встановлювати, тестувати, версіювати та розповсюджувати як єдине ціле — на відміну від окремого скрипта.

2. Чим проект відрізняється від одного Python script?

Один скрипт — це, як правило, один файл .py, який виконує конкретну задачу і не має структури, залежностей, тестів чи метаданих. Проект має чітку структуру (пакети, модулі), файл конфігурації (pyproject.toml), систему керування залежностями, віртуальне середовище, тести, документацію та контроль версій.

3. Що таке Python module?

Це окремий файл із розширенням .py, який містить Python-код (функції, класи, змінні) і може бути імпортований в іншому файлі за допомогою import.

4. Що таке Python package?

Це каталог, що містить кілька модулів і файл **init.py** (у класичному варіанті), який дозволяє Python розглядати цей каталог як єдиний імпортований пакет із власним простором імен.

5. Для чого потрібний init.py?

Він позначає каталог як Python-пакет, може виконувати ініціалізацію пакета при імпорті, а також визначати, які саме імена (класи, функції, підмодулі) експортуються назовні при імпорті пакета.

6. Що таке src-layout?

Це спосіб організації проекту, за якого вихідний код пакета розміщується не в корені проекту, а в окремому каталозі src/<назва_пакета>, а тести, конфігурація та документація — на рівень вище, поруч зі src.

7. Які переваги має src-layout?

- Унеможливилює випадковий імпорт пакета напряму з кореня проєкту без його встановлення, тому тестується саме встановлена версія.
- Чіткіше відокремлює вихідний код від конфігураційних і допоміжних файлів.
- Зменшує ризик конфліктів імен між пакетом і сторонніми модулями.
- Полегшує коректне пакування та публікацію проєкту.

8. Для чого використовується `virtual environment`?

Щоб ізолювати залежності конкретного проєкту від глобального Python та від інших проєктів — кожен проєкт має власний набір пакетів потрібних версій, що уникає конфліктів версій.

9. Як створити `.venv`?

Командою `python -m venv .venv` у корені проєкту — вона створює каталог `.venv` із власною копією інтерпретатора та інструментами для встановлення пакетів.

10. Як активувати `virtual environment`?

У Windows PowerShell — командою `.\venv\Scripts\Activate.ps1`; у Linux/macOS — командою `source .venv/bin/activate`. Після активації в рядку запрошення з'являється префікс `(.venv)`.

11. Для чого використовується `pip`?

Це стандартний менеджер пакетів Python, який дозволяє встановлювати, оновлювати та видаляти сторонні бібліотеки з PyPI чи локальних джерел.

12. Чому рекомендується `python -m pip`?

Тому що ця форма гарантовано викликає `pip` саме того інтерпретатора Python, який активний (наприклад, у віртуальному середовищі), тоді як просто `pip` може випадково посилатися на інший, глобальний Python у системі, що призводить до встановлення пакетів "не туди".

13. Для чого потрібний `pyproject.toml`?

Це єдиний стандартизований файл конфігурації сучасного Python-проєкту: він описує систему збирання (`build-system`), метадані пакета (назва, версія, залежності, точки входу) і налаштування інструментів (лінтерів, форматерів, тестових фреймворків).

14. Що містить секція `[project]`?

Основні метадані пакета: `name` (назва), `version` (версія), `description` (опис), `requires-python` (мінімальна версія Python), `dependencies` (залежності), а також підсекції на кшталт `[project.scripts]` для консольних точок входу.

15. Що таке `editable installation`?

Це встановлення пакета командою `pip install -e .`, за якого пакет не копіюється в

site-packages, а посилається на вихідні файли проєкту. Завдяки цьому будь-які зміни в коді одразу діють без повторного встановлення пакета.

16. Для чого використовується `if name == "main"`?

Ця конструкція дозволяє коду виконуватися лише тоді, коли файл запущено безпосередньо (як скрипт), а не коли його імпортовано як модуль в іншому файлі. Це відокремлює логіку запуску програми від логіки, яку варто перевикористовувати.

17. Що таке Git?

Це розподілена система контролю версій, яка відстежує зміни у файлах проєкту, дозволяє зберігати історію цих змін, повертатися до попередніх станів і спільно працювати над кодом.

18. Що таке repository?

Це сховище проєкту (каталог `.git`), у якому Git зберігає всю історію змін, гілки, коміти та метадані версійного контролю.

19. Що таке commit?

Це зафіксований "знімок" стану проєкту в певний момент часу — набір змін із унікальним ідентифікатором, автором, датою та повідомленням, що описує суть змін.

20. Для чого використовується `git status`?

Щоб побачити поточний стан робочої теки: які файли змінено, які додано до індексу (staged), а які ще не відстежуються Git.

21. Для чого `git add`?

Щоб додати зміни у файлах до індексу (staging area) — тобто позначити, які саме зміни увійдуть до наступного коміту.

22. Для чого `git commit`?

Щоб зафіксувати всі зміни, додані через `git add`, як окремий запис в історії репозиторію разом із повідомленням, що пояснює суть цих змін.

23. Що показує `git log`?

Історію комітів репозиторію: ідентифікатори комітів, авторів, дати та повідомлення — тобто хронологію розвитку проєкту.

24. Для чого потрібен `.gitignore`?

Щоб вказати Git, які файли та каталоги не потрібно відстежувати й додавати до репозиторію — зазвичай службові, тимчасові, згенеровані або приватні файли (кеш, віртуальне середовище, секрети).

25. Чому `.venv` не потрібно комітити?

Тому що це середовище є суто локальним і залежить від конкретної операційної

системи та шляху; воно легко відтворюється командою `python -m venv .venv` та встановленням залежностей, а зберігання його в репозиторії лише роздуває розмір репозиторію та може спричиняти конфлікти між платформами.

26. Для чого потрібний README?

Це базова документація проєкту — файл, який пояснює, що робить проєкт, як його встановити, налаштувати та запустити, яку має структуру, і за потреби — як зробити внесок у розробку.

27. Що таке type hints?

Це анотації типів у сигнатурах функцій і змінних (наприклад, `def avg_price(tech_list: list[Tech]) -> float:`), які явно вказують очікувані типи даних. Вони не змінюють поведінку програми під час виконання, але покращують читабельність коду та дозволяють статичним аналізаторам (муру) виявляти помилки типів заздалегідь.

28. Для чого програму розділяють на modules?

Щоб покращити структурованість, читабельність і підтримуваність коду: кожен модуль відповідає за окрему логічну частину функціоналу, що спрощує тестування, повторне використання коду та командну роботу.

29. Що таке business logic?

Це основна логіка застосунку, що реалізує правила та операції предметної області (наприклад, пошук, сортування, обчислення вартості техніки) — на відміну від допоміжного коду (введення/виведення, інтерфейс, запуск програми).

30. Чому потрібно відокремлювати application entry point від business logic?

Тому що це робить бізнес-логіку незалежною від способу запуску програми: її можна тестувати окремо, повторно використовувати в інших інтерфейсах (CLI, веб, API) і легше підтримувати, оскільки зміни в способі запуску (наприклад, у `main.py`) не зачіпають саму логіку обробки даних.

Висновок: Виконуючи цю лабораторну роботу, можна ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки, формувати структуру проєкту за принципом `src-layout`, працювати з модулями та пакетами Python, створювати файл `pyproject.toml`, використовувати Git для контролю версій та оформлювати базову документацію проєкту.

